

On Drone Communication General Study

The mission control unit consists of Python scripts that run within a mission control computer such as the Nvidia Jetson Nano and establish a serial connection with Pixhawk via the MAVLink protocol. These scripts analyze the data obtained through image processing algorithms and process the drone status information from Pixhawk. Then, it calculates the necessary commands and transmits them to Pixhawk.

Here, the input is the information obtained by image processing algorithms and the drone status information (such as location, speed, altitude, battery status) received from Pixhawk. As output, control commands (such as position, speed, angle adjustments) transmitted to Pixhawk are generated.

Some software and libraries are used for the operation of this system:

ArduPilot: *It is the flight control software that runs on Pixhawk. By installing this software on Jetson Nano, we will communicate with Pixhawk via the MAVLink protocol.*

MAVLink Protocol: *It is the communication protocol between Jetson Nano and Pixhawk. MAVLink provides a low-latency and reliable data transfer.*

DroneKit: *It is a Python library that allows us to communicate with Pixhawk through the MAVLink protocol. With DroneKit, we can read the drone's telemetry information and send commands.*

Gazebo Simulation Environment: *It is a simulation software used to test and visually observe our scripts, we can try the code and reverses we make from here. <https://ubuntu.com/download/desktop>*

Ubuntu was chosen...

First of all, let's set up the pip with the command below

```
sudo apt-get update sudo apt-get  
upgrade sudo apt update sudo apt install  
python3-pip python3-dev sudo apt-get  
install python3-pip
```

Next, we will install the ArduPilot software. For this, we will use the site given below. It is important to read the steps carefully because some of them involve the actions you need to take if you get an error, while others are steps that need to be taken directly. If you follow the steps carefully, you can understand what actions you need to take in which situations.

<https://ardupilot.org/dev/docs/building-setup-linux.html>

This setup enables ArduPilot to operate in a virtual network. Thus, we can perform tests in a simulation environment as if there was a Pixhawk, without a physical Pixhawk connection.

The installation on the web address was done respectively

Step by step together:

Ubuntu

Linux/Ubuntu users can install with apt :

- `sudo apt-get update`
- `sudo apt-get install git`
- `sudo apt-get install gitk git-gui`

or the main project repo:

```
git clone https://github.com/ArduPilot/ardupilot
cd ardupilot
git submodule update --init --recursive
```

git clone https://github.com/ArduPilot/ardupilot

***cd ardupilot git submodule update --init --
recursive***

Tools/environment_install/install-prereqs-ubuntu.sh -y

ArduPilot is an open-source and flexible autonomous vehicle software that can control unmanned air, land, and sea vehicles.

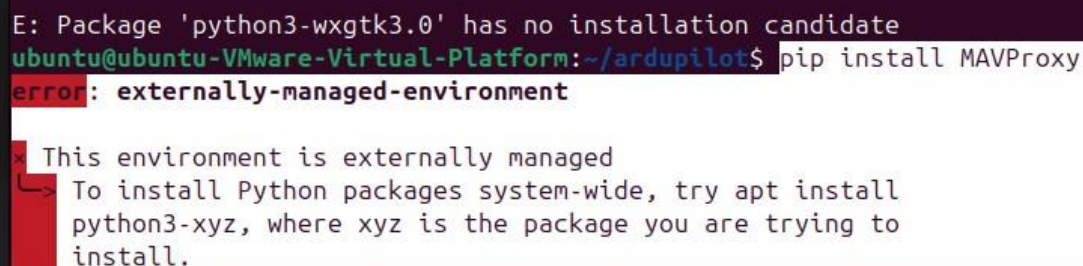
Thanks to its advanced flight control algorithms, it can be used for both manual and fully autonomous missions.

Now it's time for MAVproxy

MAVProxy is a lightweight and modular ground control station (GCS) software developed for vehicles operating with the MAVLink protocol. While it was originally developed for ArduPilot systems, it can be used with all autonomous vehicles that support MAVLink.

sudo apt-get install python3-dev python3-opencv python3-wxgtk3.0 python3-pip python3-matplotlib python3-pygame python3-lxml python3-yaml

Then : pip install MAVProxy



```
E: Package 'python3-wxgtk3.0' has no installation candidate
ubuntu@ubuntu-VMware-Virtual-Platform:~/ardupilot$ pip install MAVProxy
error: externally-managed-environment

x This environment is externally managed
  To install Python packages system-wide, try apt install
  python3-xyz, where xyz is the package you are trying to
  install.
```

We can solve this error in two ways

1 Virtual environment

2 --break-system-packages Force Install Using

Which can damage the system, the suggestion would be a virtual environment. venv Installation : sudo apt install python3-venv

Creating a Virtual Environment: python3 -m venv mavproxy_env

Activating the Virtual Environment : source mavproxy_env/bin/activate

So now it is enough to do `source mavproxy_env/bin/activate` when opening the ones we got this error continued from this side...

```
Setting up python3-venv (3.12.3-0ubuntu2) ...
ubuntu@ubuntu-VMware-Virtual-Platform:~/ardupilot$ python3 -m venv mavproxy_env
ubuntu@ubuntu-VMware-Virtual-Platform:~/ardupilot$ source mavproxy_env/bin/activate
(mavproxy_env) ubuntu@ubuntu-VMware-Virtual-Platform:~/ardupilot$ pip install MAVProxy
Collecting MAVProxy
  Using cached MAVProxy-1.8.71-py3-none-any.whl.metadata (2.0 kB)
Collecting pymavlink>=2.4.14 (from MAVProxy)
  Using cached pymavlink-2.4.43-py3-none-any.whl.metadata (6.4 kB)
Collecting pyserial>=3.0 (from MAVProxy)
  Downloading pyserial-3.5-py2.py3-none-any.whl.metadata (1.6 kB)
Collecting numpy (from MAVProxy)
  Using cached numpy-2.2.4-cp312-cp312-macosx_2_17_x86_64_machd...
```

As you can see, the problem has been solved and the installation has begun...

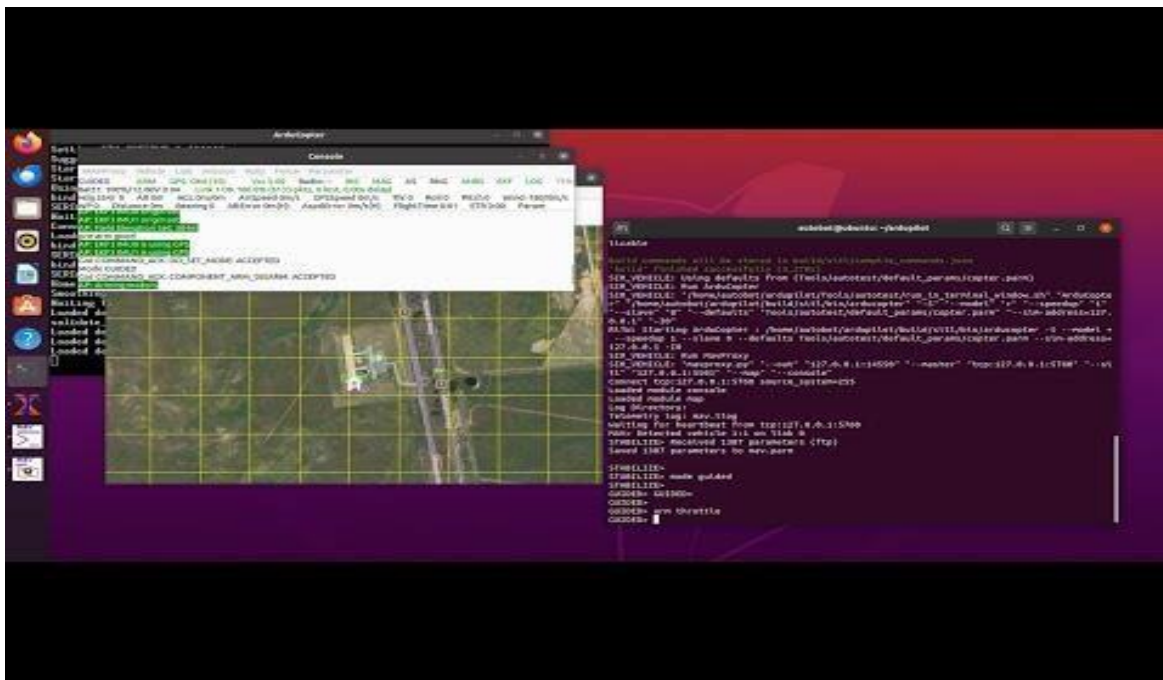
For more information: <https://ardupilot.org/mavproxy/>

<https://ardupilot.org/dev/docs/sitl-with-gazebo.html>

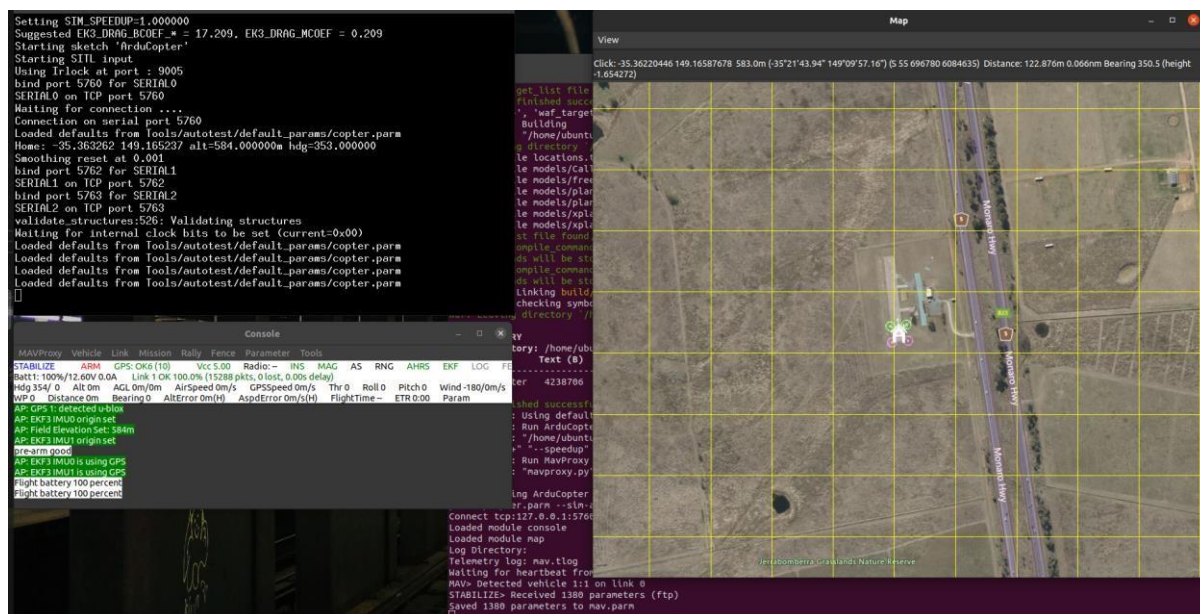
By setting up the gazebo simulator with ArduPilot, if you see a drone standing on the runway when the simulation starts, the installation is successful. You can find detailed installation steps [at this link](#) .

<https://www.youtube.com/watch?v=gslpbWOkG5o>

<https://www.youtube.com/watch?v=r35vcFrVFjQ>



The men here were watched



Sardupilot SITL

We Launched ArduCopter Simulation:

We simulated ArduCopter with the `sim_vehicle.py` command.

MAVProxy Connection Established:

Communicated with ArduPilot via TCP 127.0.0.1:5760.

The message "Waiting for heartbeat" indicated that the simulation was waiting for a connection with the flight controller.

"Detected vehicle 1:1 on link 0" → Vehicle 1 (ID: 1) has been successfully identified.

Parameters Loaded:

ArduCopter took 1380 parameters and saved them in the `mav.parm` file.

These parameters are the flight controller's settings (e.g., PID values, motor settings, etc.).

You are in STABILIZED Mode:

STABILIZE> → The simulation is currently in manual stabilization mode.

In this mode, the movement of the drone is provided by manual control (RC controller or commands).

Code entered to be checked in order and its meanings :

mode GUIDED

Puts the drone in GUIDED mode.

The GUIDED mode allows controlling the drone with MAVLink commands.

Instead of manual control, it takes commands from external sources such as ground station (GCS) or Python scripts.

arm throttle

It arms the drone's motors to run (ARM does).

The drone is ready to start its engines and take off.

*If the drone's security checks (GPS, IMU, battery, etc.) are not completed, ARM may fail **takeoff 10***

He tells the drone to rise up to 10 meters.

It should be in GUIDED mode!

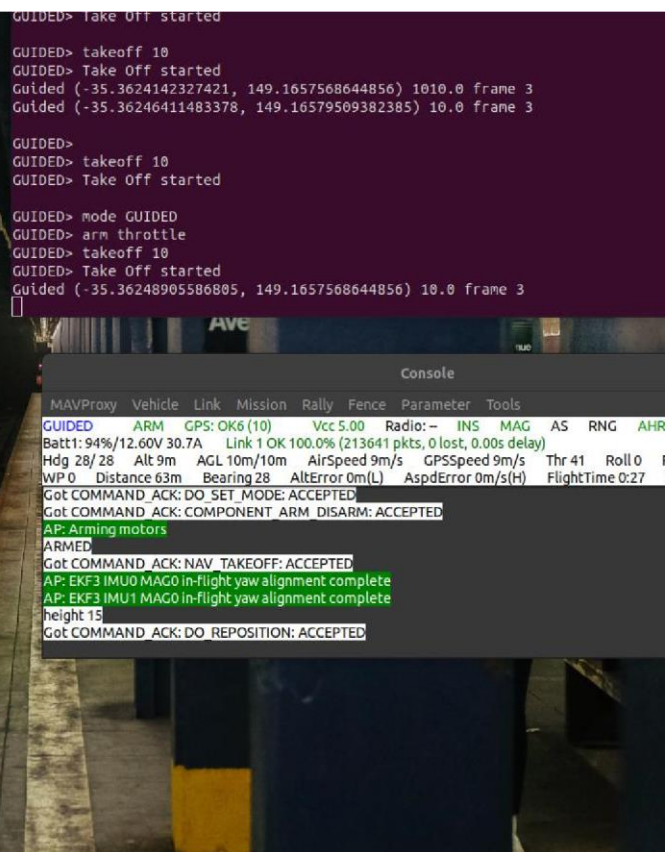
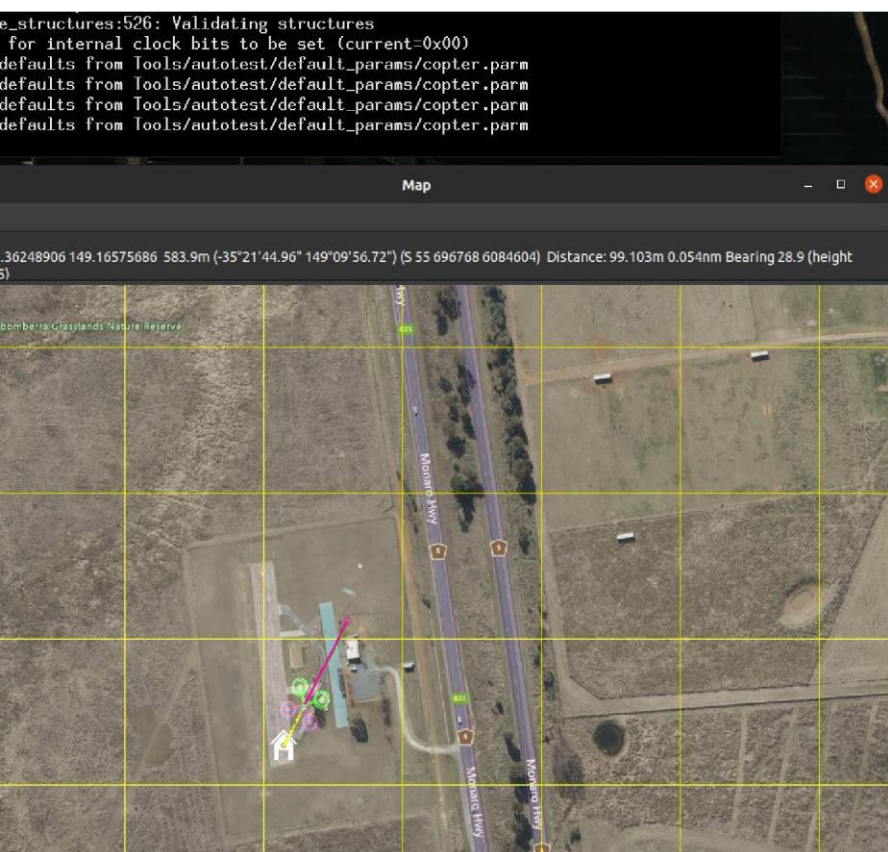
If the motors are running, the drone automatically rises to 10 meters.

If everything is fine, we get the following message:

Takeoff to 10 meters

So in summary: 1) Switch to GUIDED mode → 2) Start the engines →

3) 10 meters elevation



*mode **GUIDED***

arm throttle

takeoff 10

AND SUCCESSFUL...

*mode **LAND***

Puts the drone in automatic landing mode.

In this mode, the drone descends vertically from its location and lands on the ground.

*mode **LOITER***

The drone remains stationary in its location with GPS support.

Even if there is wind or other external factors, it tries to maintain its position.

Steering can be done with a joystick or RC controller.

arm throttle

rc 3 1600 / rc 3 1700 / rc 3 1500 (Throttle - Height Control) These commands change the Throttle (RC Channel 3 - Throttle) values.

Meaning:

rc 3 1600 → Increases engine power, drone rises. rc

3 1700 → Rises fast by giving more power. rc 3 1500

→ Balances, neither ascends nor descends.

rc 4 1600 / rc 4 1500 → (Yaw - Direction Change)

These commands are for RC Channel 4 (Yaw - Direction Control).

Meaning:

rc 4 1600 → Clockwise rotation begins.

rc 4 1500 → Stops spinning.

rc 2 1400 / rc 2 1500 → (Pitch - Forward and Backward Movement) These commands are for RC Channel 2 (Pitch - Forward/Backward Movement).

Meaning:

rc 2 1400 → The drone moves forward.

rc 2 1500 → Stops movement (neutral position).

rc 1 1400 / rc 1 1600 → (Roll - Move Right and Left) These commands are for RC Channel 1 (Roll - Move Right/Left).

Meaning:

rc 1 1400 → The drone moves left. rc 1

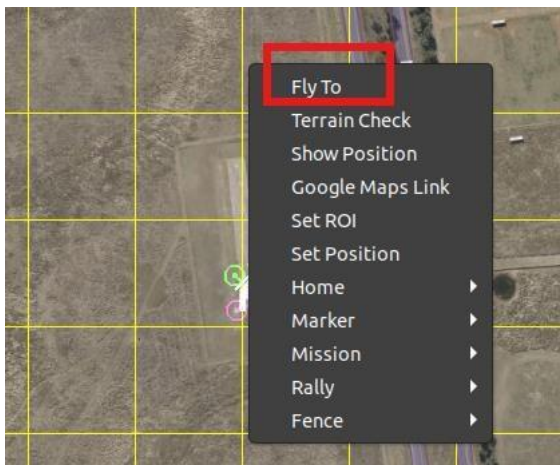
1600 → The drone moves to the right.

So in summary:

*mode guided : mode opens the drone command arm throttle :
starts the engines but stops if we wait too long*

takeoff 10 : rise 10 m

After that, right-click on the map and

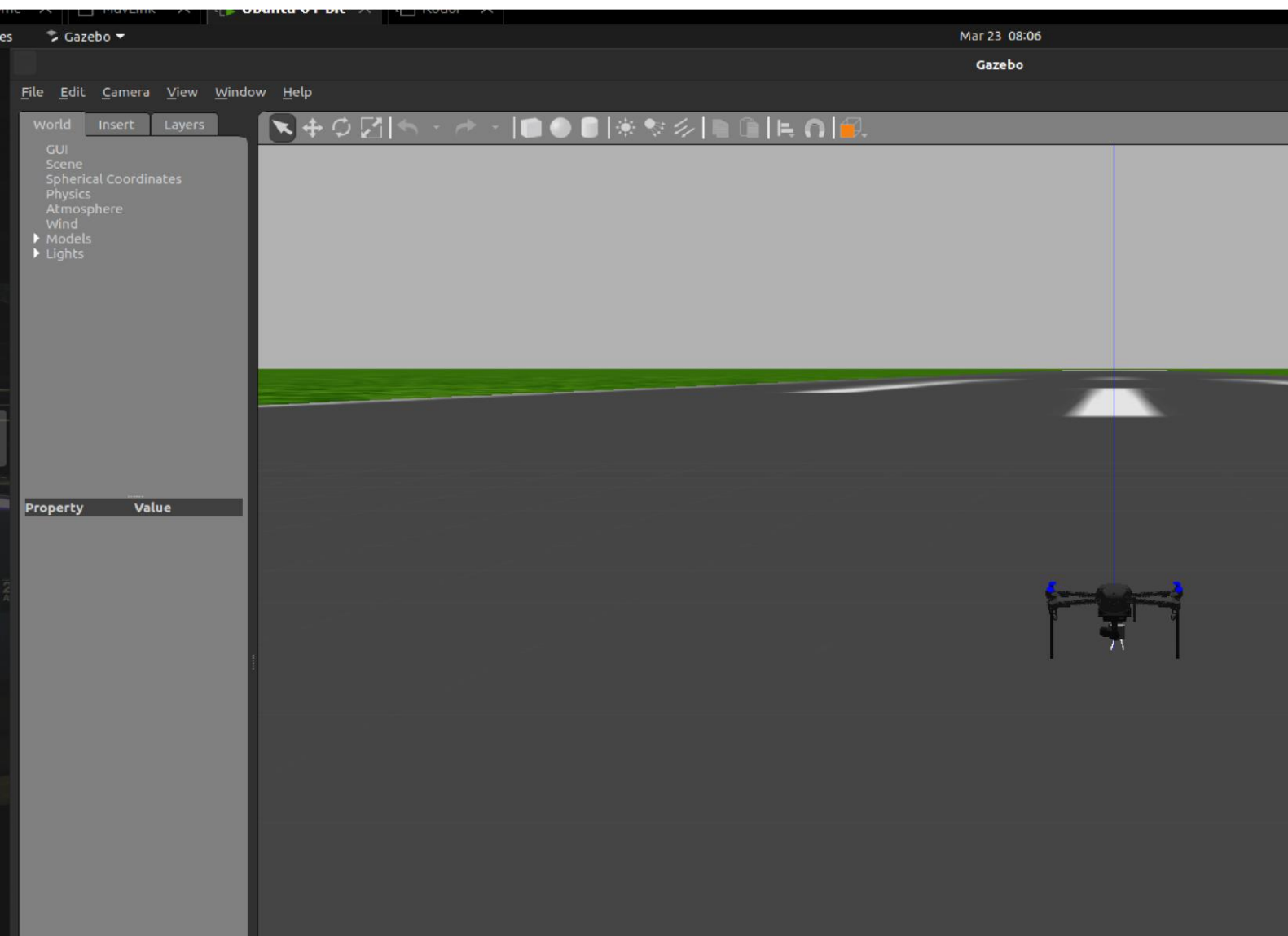


Wherever we select and click, we can send the drone there.

..... and similar codes have been tried....

And if you still have problems with the installation:

<https://www.emrehaninci.com/drone-development-environment-installation/> And this is installed, we would start



<https://www.youtube.com/watch?v=t3hqj0ujZnc&list=PLdxBoQOSnnGTp25xrsl2mt4Xt66iIOwzW&index=6>

Despite these, a small error was received, but it happened when I did these

3. Yükleme Sonrasında MAVProxy'yi PATH'e Ekleyin

Bazı durumlarda, yükleme sonrası `mavproxy.py` doğrudan çalışmaz. MAVProxy'nin nereye yüklendiğini bulmak için:

```
bash
```

Kopyala

```
which mavproxy.py
```

Eğer bir yol döndürmezse, elle ekleyebilirsiniz:

```
bash
```

Kopyala

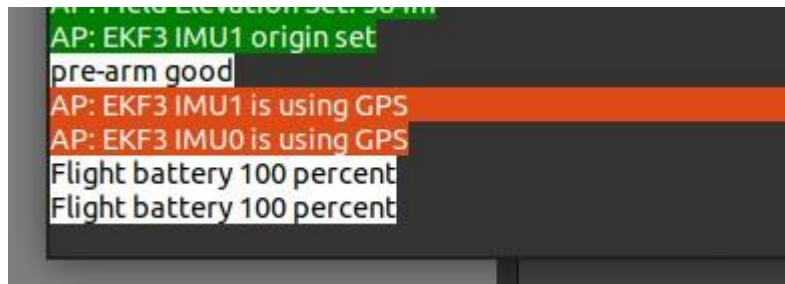
```
export PATH=$PATH:$HOME/.local/bin
```

Bu değişikliği kalıcı hale getirmek için:

```
bash
```

Kopyala

```
echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc  
source ~/.bashrc
```



After these two warnings, send a code to the drone


```

Loaded module map
MAV> mode guided
STABILIZE> GUIDED> arm throttle
GUIDED> takeoff 10
GUIDED> Take Off started
position 10 0 0
GUIDED> x:10.000000, y:0.000000, z:0.000000

```

```

:5501" "--map" "--console"
Connect tcp:127.0.0.1:5760 source_system=255
Loaded module console
Loaded module map
Log Directory:
Telemetry log: mav.tlog
Waiting for heartbeat from tcp:127.0.0.1:5760
MAV> Detected vehicle 1:1 on link 0
STABILIZE> Received 1414 parameters (ftp)
Saved 1414 parameters to nav.parm
mode guided
STABILIZE> arm throttle
GUIDED> GUIDED> takeoff 10
GUIDED> Take Off started

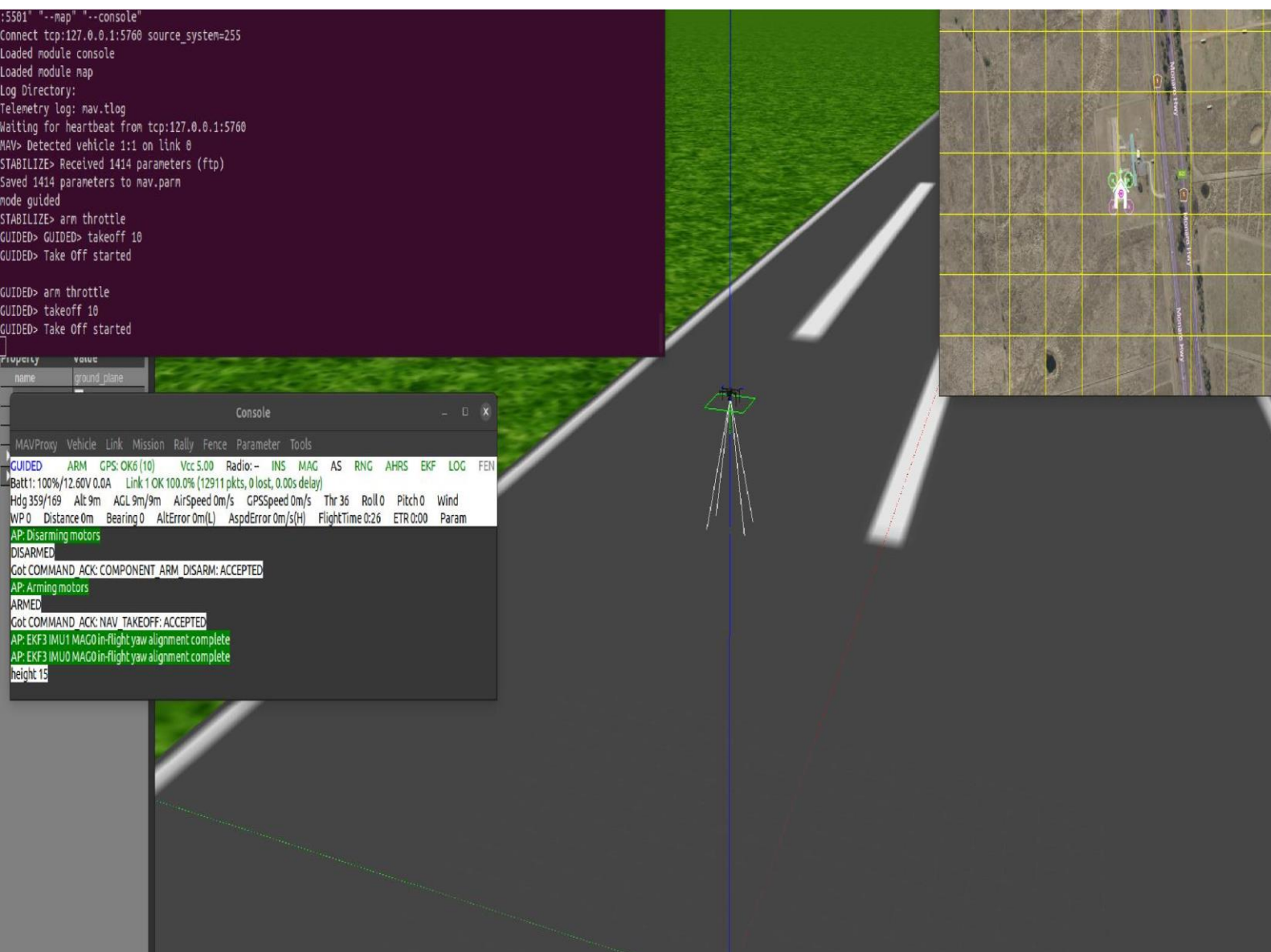
GUIDED> arm throttle
GUIDED> takeoff 10
GUIDED> Take Off started

```

Property	Value
name	ground_plane

Console

MAVProxy Vehicle Link Mission Rally Fence Parameter Tools
GUIDED ARM GPS: OK6 (10) Vcc 5.00 Radio:-- INS MAG AS RNG AHRS EKF LOG FEN
Batt1: 100%/12.60V 0.0A Link 1 OK 100.0% (12911 pkts, 0 lost, 0.00s delay)
Hdg 359/169 Alt 9m AGL 9m/9m AirSpeed 0m/s GPSSpeed 0m/s Thr 36 Roll 0 Pitch 0 Wind
WP 0 Distance 0m Bearing 0 AltError 0m(L) AspdError 0m/s(H) FlightTime 0:26 ETR 0:00 Param
AP: Disarming motors
DISARMED
Got COMMAND ACK: COMPONENT_ARM_DISARM: ACCEPTED
AP: Arming motors
ARMED
Got COMMAND ACK: NAV TAKEOFF: ACCEPTED
AP: EKF3 IMU1 MAG0 in-flight yaw alignment complete
AP: EKF3 IMU0 MAG0 in-flight yaw alignment complete
height 15



```
ubuntu@ubuntu: ~/ardupilot_gazebo/build
27.0.0.1 -I0
SIM_VEHICLE: Run MavProxy
SIM_VEHICLE: "mavproxy.py" "--retries" "5" "--out" "127.0.0.1:14550" "--master" "tcp:127.0.0.1:5760" "--sitl"
:5501" "--map" "--console"
Connect tcp:127.0.0.1:5760 source_system=255
Loaded module console
Loaded module map
Log Directory:
Telemetry log: mav.tlog
Waiting for heartbeat from tcp:127.0.0.1:5760
MAV> Detected vehicle 1:1 on link 0
STABILIZE> Received 1414 parameters (ftp)
Saved 1414 parameters to mav.parm
mode guided
STABILIZE> arm throttle
GUIDED> GUIDED> takeoff 10
GUIDED> Take Off started

GUIDED> arm throttle
GUIDED> takeoff 10
GUIDED> Take Off started
position 10 0 0
GUIDED> x:10.000000, y:0.000000, z:0.000000

Property      value
name          ground plane
```

Console

MAVProxy	Vehicle	Link	Mission	Rally	Fence	Parameter	Tools
GUIDED	ARM	GPS: OK6 (10)	Vcc 5.00	Radio: -	INS	MAG	AS RNG AHRS EKF LOG FEN
Batt1: 100%/12.60V 0.0A Link 1 OK 100.0% (25873 pkts, 0 lost, 0.00s delay)							
Hdg 359/245 Alt 10m AGL 10m/9m AirSpeed 0m/s GPSSpeed 0m/s Thr 36 Roll 0 Pitch 0 Wind							
WP 0 Distance 0m Bearing 148 AltError 0m(H) AspdError 0m/s(H) FlightTime 2:19 ETR 0:00 Param							
Got COMMAND ACK: COMPONENT_ARM_DISARM: ACCEPTED							
AP: Arming motors							
ARMED							
Got COMMAND ACK: NAV TAKEOFF: ACCEPTED							
AP: EKF3 IMU1 MAG0 in-flight yaw alignment complete							
AP: EKF3 IMU0 MAG0 in-flight yaw alignment complete							
height 15							
Flight battery 100 percent							
Flight battery 100 percent							

Opening gezebo first `ubuntu@ubuntu:~/ardupilot_gazebo/build$`
`gazebo --verbose worlds/iris_arducopter_runway.world`

Then the Arddu pilot is employed

ubuntu@ubuntu:~/ardupilot/ArduCopter\$

.. /Tools/autotest/sim_vehicle.py -f gazebo-iris --console --map

```
GUIDED> arm throttle
GUIDED> takeoff 10
GUIDED> Take Off started
position 10 0 0
GUIDED> x:10.000000, y:0.000000, z:0.000000
position -10 0 0
GUIDED> x:-10.000000, y:0.000000, z:0.000000
position 0 10 0
GUIDED> x:0.000000, y:10.000000, z:0.000000
position 0 -10 0
GUIDED> x:0.000000, y:-10.000000, z:0.000000
position 0 0 5
GUIDED> x:0.000000, y:0.000000, z:5.000000
position 0 0 -5
GUIDED> x:0.000000, y:0.000000, z:-5.000000
param show
GUIDED> ACRO_BAL_PITCH    1.0
ACRO_BAL_ROLL            1.0
ACRO_OPTIONS              0
ACRO_RP_EXPO              0.300000001192092896
ACRO_RP_RATE              360.0
ACRO_RP_RATE_TC           0.0
```

position X Y Z Command Explanation

X (forward/backward movement)

Positive (+10) → moves forward 10 meters.

Negative (-10) → goes back 10 meters.

Y (moving right/left)

Positive (+10) → goes 10 meters to the right.

Negative (-10) → goes 10 meters to the left.

Z (altitude control, not altitude change)

Positive (+10) → descend by 10 meters (because Z represents altitude relative to sea level).

Negative (-10) → rises by 10 meters.

Param Show

It allows you to see all the parameters.

If you want to see a specific parameter: param show

```
WPNAV_SPEED_DN 150.0
WPNAV_SPEED_UP 250.0
WPNAV_TER_MARGIN 10.0
WP_NAVALT_MIN 0.0
WP_YAW_BEHAVIOR 2
WVANE_ENABLE 0
ZIGZ_AUTO_ENABLE 0
mode rtl
GUIDED> [ ]
```

Console

MAVProxy Vehicle Link Mission Rally Fence Parameter Tools

RTL ARM GPS: OK6 (10) Vcc 5.00 Radio: -- INS MAG AS RNG AHRS EKF LOG FEN R

Batt1: 100%/12.60V 0.0A Link 1 OK 100.0% (51142 pkts, 0 lost, 0.00s delay)

Hdg 179/243 Alt 5m AGL 5m/5m AirSpeed 0m/s GPSSpeed 0m/s Thr 35 Roll 0 Pitch 0 Wind

WP 0 Distance 0m Bearing 128 AltError 0m(L) AspdError 0m/s(H) FlightTime 4:56 ETR 0:00 Param

height 15

Flight battery 100 percent

Flight battery 100 percent

Flight battery 100 percent

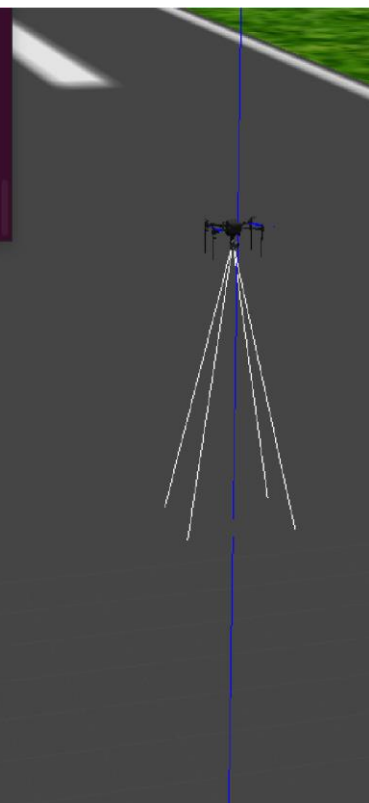
Flight battery 100 percent

Got COMMAND ACK: DO SET_MODE: ACCEPTED

Mode RTL

Flight battery 100 percent

AP: PreLand: Failsafe Measures



[PARAM_ADI]

Baris Adigüzel